

HITO 0 — PITCH

Sistema de Gestión de Turnos

Fecha de entrega: martes 15 de septiembre de 2026 · Repositorio: github.com/LeandroLicata/proyecto-integrador-turnos

INTEGRANTES

Integrante	Responsabilidad principal
Leandro Licata	main.py — menú, flujo e integración de módulos
Federico Cabrera	estructuras.py — cola de turnos e historial de atenciones
Gonzalo Tapia	persistencia.py — lectura y escritura de JSON, CSV y TXT
Gustavo Di Paola	utils.py y estadísticas.py — validaciones, búsquedas y métricas

Las pruebas unitarias y la documentación (README.md, PROMPTS.md e informe) se reparten entre los cuatro integrantes. Cada uno versiona su propio trabajo con commits a su nombre.

TEMA ELEGIDO

Sistema de gestión de turnos (perfil Desarrollo de Software). Aplicación de consola en Python que administra la atención por orden de llegada en un consultorio o comercio: registra a los pacientes, los coloca en una cola de espera, lleva el historial de atenciones y produce estadísticas de demanda. Resuelve un problema concreto: hoy ese registro suele llevarse en papel o en una planilla suelta, sin trazabilidad ni datos para dimensionar el horario de atención.

ALCANCE FUNCIONAL (MENÚ PRINCIPAL)

1. **Registrar paciente** — alta con validación de DNI, nombre y teléfono.
2. **Asignar turno** — encola al paciente por orden de llegada, con fecha y hora de registro.
3. **Atender turno** — desencola al siguiente, registra la hora de atención y lo pasa al historial.
4. **Consultas y búsquedas** (submenú) — por DNI, por nombre, por rango de fechas y cola actual.
5. **Modificar o cancelar un turno** — edición de datos y baja con motivo.
6. **Estadísticas del período** — métricas sobre los turnos atendidos.
7. **Exportar reporte diario a CSV.** **0. Salir.**

DISEÑO TÉCNICO

- **Modelado sin POO:** listas de diccionarios (lista_pacientes, cola_turnos, historial_atenciones) y funciones constructoras como crear_turno() con ID autoincremental.
- **Estructura central:** una cola (FIFO) para los turnos en espera, implementada sobre listas, coherente con la atención por orden de llegada.
- **Persistencia:** turnos.json (estado principal, se carga al iniciar y se guarda tras cada operación), reporte_AAAA-MM-DD.csv (reporte diario exportable) y log_operaciones.txt (bitácora con fecha y hora de cada acción, vía datetime). Todo con bloque with y try/except.
- **Búsqueda y ordenamiento:** filtrado por criterio sobre la lista de diccionarios y ordenamiento propio (inserción) para listar el historial por fecha o por tiempo de espera.
- **Estadísticas (mínimo 2):** tiempo promedio de espera, atenciones por franja horaria, porcentaje de turnos cancelados o ausentes y ranking de los días de mayor demanda.
- **Módulos:** main.py, estructuras.py, persistencia.py, utils.py y estadísticas.py. Nomenclatura snake_case en todo el proyecto, según PEP 8.

EJE DE INVESTIGACIÓN

Opción D — Visualización de datos con matplotlib. Es el eje que mejor aprovecha las estadísticas del sistema: permite graficar la distribución de atenciones por franja horaria y la evolución del tiempo de espera, exportando los gráficos como PNG. Se investigará en el informe y se implementará en código para sumar el puntaje adicional.

PLAN DE TRABAJO

Clase	Fecha	Actividad
1	15/09	Hito 0 — Pitch. Tema, integrantes, plan y repositorio creado.
2	22/09	main.py con el menú en bucle y persistencia.py con JSON. Registrar paciente operativo.
3	29/09	Cola de turnos, atención, búsquedas, ordenamiento y primeras estadísticas.
4	06/10	Hito 1 — v1.0. Menú + persistencia + una funcionalidad completa.
5	13/10	Exportación CSV, log TXT, pruebas unitarias y PROMPTS.md.
6	20/10	Refactorización, gráficos con matplotlib, informe PDF y grabación del video.
7	27/10	Hito 2 — Final. Entrega completa y defensa oral.

TRAZABILIDAD

Se adopta la **Opción A — control de versiones con Git**. El repositorio es público y el historial refleja el avance incremental de cada integrante, con un mínimo de 15 commits descriptivos en español. Los commits que incorporen código asistido por IA lo indican en el mensaje, y el detalle de cada uso queda registrado en PROMPTS.md.